

COL775 Assignment 1 - Part 1

Resnet and Normalisation Schemes

Yash Bansal (2022CS51133)

March 27, 2026

Contents

1	Introduction	2
1.1	Experimental Overview	2
1.2	Model Availability	2
2	Implementation and Design Choices	3
2.1	ResNet-18 Architecture	3
2.2	Pre-Norm vs Post-Norm	3
2.3	Weight Initialization	3
2.4	Group Normalization Configuration	3
2.5	Normalization Schemes	3
3	Hyperparameter Tuning	5
4	Impact of Normalisation	6
4.1	Training Curves	6
4.2	Quantitative Results	12
4.3	Analysis	13
4.3.1	Baseline vs Custom Batch Normalization	13
4.3.2	Batch-Instance Normalization	13
4.3.3	Instance Normalization	13
4.3.4	Layer Normalization and Group Normalization	13
4.3.5	No Normalization	14
4.3.6	Overall Observations	14
5	Batch Size Sensitivity	15
5.1	Setup	15
5.2	Training Curves	15
5.3	Quantitative Results	16
5.4	Analysis	17
6	Grad-CAM Visualizations	18
6.1	Visualizations	18
6.2	Analysis	20
6.2.1	Best Performing Classes	20
6.2.2	Worst Performing Classes	21
6.2.3	General Observations	21

1 Introduction

This assignment explores the ResNet architecture and the impact of various normalization schemes on image classification performance. We implement ResNet-18 from scratch and evaluate it on a 100-class subset of ImageNet-1k, systematically replacing Batch Normalization with five alternative schemes to study their effect on training dynamics and generalization. We use a 100-class subset of ImageNet-1k containing 680 training and 50 validation images per class at various resolutions.

1.1 Experimental Overview

We conduct the following experiments:

1. **ResNet-18 Baseline:** Implement ResNet-18 from scratch with PyTorch’s built-in `nn.BatchNorm2d` as a performance reference.
2. **Normalization Scheme Comparison:** Replace Batch Normalization with five custom-implemented schemes, namely Instance Normalization (IN), Batch-Instance Normalization (BIN), Layer Normalization (LN), Group Normalization (GN), and No Normalization (NN), and compare performance across all variants.
3. **Batch Size Sensitivity:** Empirically verify GN’s robustness to small batch sizes (batch size = 8) compared to BN, which is known to degrade under such conditions.
4. **Grad-CAM Analysis:** Use Gradient-weighted Class Activation Mapping to visualize and interpret model predictions across 5 selected classes, analyzing both correct and incorrect classifications.

1.2 Model Availability

Trained model weights are available at:

<https://drive.google.com/drive/folders/1LYFPKcwAldIZjtbnduwPeZrh4b8zfSfX?usp=sharing>

2 Implementation and Design Choices

2.1 ResNet-18 Architecture

We implement ResNet-18 from scratch following the original paper. The architecture consists of the following components:

1. **Initial Block:** A 7×7 convolution with 64 filters, stride 2, and padding 3, followed by a normalization layer, ReLU activation, and a 3×3 max pool with stride 2, reducing the spatial dimensions from 224×224 to 56×56 .
2. **Residual Layers:** Four sequential residual layers (`layer1` to `layer4`), each containing two residual blocks. Channel widths are 64, 128, 256, and 512 respectively. Downsampling (stride 2) occurs at the first block of `layer2`, `layer3`, and `layer4`.
3. **Skip Connections:** When the input and output dimensions differ (due to stride or channel mismatch), a 1×1 projection convolution followed by a normalization layer is used in the shortcut path.
4. **Global Average Pooling:** An adaptive average pooling layer reduces each feature map to 1×1 , yielding a 512-dimensional vector.
5. **Fully Connected Layer:** A linear layer maps the 512-dimensional representation to 100 class logits.

2.2 Pre-Norm vs Post-Norm

We use **post-normalization**, where the normalization layer is applied after each convolution and before the ReLU activation, exactly as described in the original ResNet paper. Specifically, within each residual block the order is: Conv $\rightarrow$ Norm $\rightarrow$ ReLU $\rightarrow$ Conv $\rightarrow$ Norm, with ReLU applied after the residual addition. This is the standard formulation and ensures that the residual branch adds unnormalized activations directly to the skip connection, preserving the identity mapping property that is central to ResNet’s design.

2.3 Weight Initialization

All convolutional and linear layers are initialized using Kaiming Normal initialization with `fan_out` mode and ReLU nonlinearity, which is well-suited for networks with ReLU activations. Normalization layer weights (`gamma`) are initialized to 1 and biases (`beta`) to 0.

2.4 Group Normalization Configuration

For Group Normalization, we use 32 groups as the default, which is a standard choice for ResNet-style architectures and divides evenly into all channel widths used (64, 128, 256, 512).

2.5 Normalization Schemes

All normalization variants are implemented as subclasses of `torch.nn.Module` and share a common interface accepting `num_features` as input, making them drop-in replacements within the same architecture. The normalization dimensions are summarized below:

- **Batch Norm (BN):** Normalizes over the batch and spatial dimensions (N, H, W) per channel. Maintains running mean and variance for inference.

- **Instance Norm (IN):** Normalizes over spatial dimensions (H, W) per sample per channel.
- **Batch-Instance Norm (BIN):** Interpolates between BN and IN outputs using a learnable per-channel parameter $\rho \in [0, 1]$, clamped to ensure valid blending.
- **Layer Norm (LN):** Normalizes over all dimensions (C, H, W) per sample.
- **Group Norm (GN):** Divides channels into 32 groups and normalizes within each group over $(C/G, H, W)$ per sample.
- **No Norm (NN):** Identity mapping with no normalization applied.

3 Hyperparameter Tuning

We conduct an initial hyperparameter search using the Baseline (PyTorch `nn.BatchNorm2d`) model with a fixed batch size of 128 over 100 epochs, varying the optimizer, learning rate schedule, and data augmentation strategy. Table 1 summarizes the validation accuracy for each configuration.

Table 1: Hyperparameter tuning results (Baseline BN model, batch size 128, 100 epochs)

Configuration	Train Acc. (%)	Val Acc. (%)
SGD (lr=0.05, momentum=0.9, no scheduling, basic aug.)	93.10	78.28
AdamW (lr=1e-3, wd=1e-4, basic aug.)	91.51	77.00
Adam (lr=1e-3, wd=1e-4, basic aug.)	85.52	80.04
Adam + advanced aug. + label smoothing	75.60	82.18

The best configuration uses the **Adam optimizer** with learning rate 10^{-3} and weight decay 10^{-4} , paired with a Cosine Annealing scheduler ($T_{\max}=100$, $\eta_{\min}=10^{-4}$), **label smoothing** of 0.1 in the cross-entropy loss, and the following augmentation pipeline:

- **RandomResizedCrop** (224, scale (0.08, 1.0)): Crops a random region of the image and resizes it, providing scale and aspect ratio variability.
- **RandomHorizontalFlip** (p=0.5): Standard flip augmentation.
- **TrivialAugmentWide**: A state-of-the-art single-policy augmentation strategy designed for ImageNet that randomly selects and applies one of a wide range of augmentation operations per image.
- **RandomErasing** (p=0.1): Randomly erases a rectangular patch from the image, encouraging the model to not over-rely on localized features.

This configuration is fixed for all subsequent experiments across all normalization variants to ensure a fair comparison.

It is worth noting that the best configuration shows a lower train accuracy (75.60%) compared to validation accuracy (82.18%), which may appear counterintuitive. This is expected behavior and is a direct consequence of the aggressive augmentation pipeline: during training, every image is subject to random cropping, TrivialAugmentWide transformations, and random erasing, making the training task considerably harder. At validation, no such augmentations are applied, so the model sees cleaner images that are easier to classify correctly. The lower train accuracy therefore reflects the difficulty of the augmented training task rather than underfitting, and the higher validation accuracy confirms that the augmentation is successfully improving generalization.

4 Impact of Normalisation

4.1 Training Curves

Figures 1 through 7 show the training and validation curves for all seven model variants across all metrics.

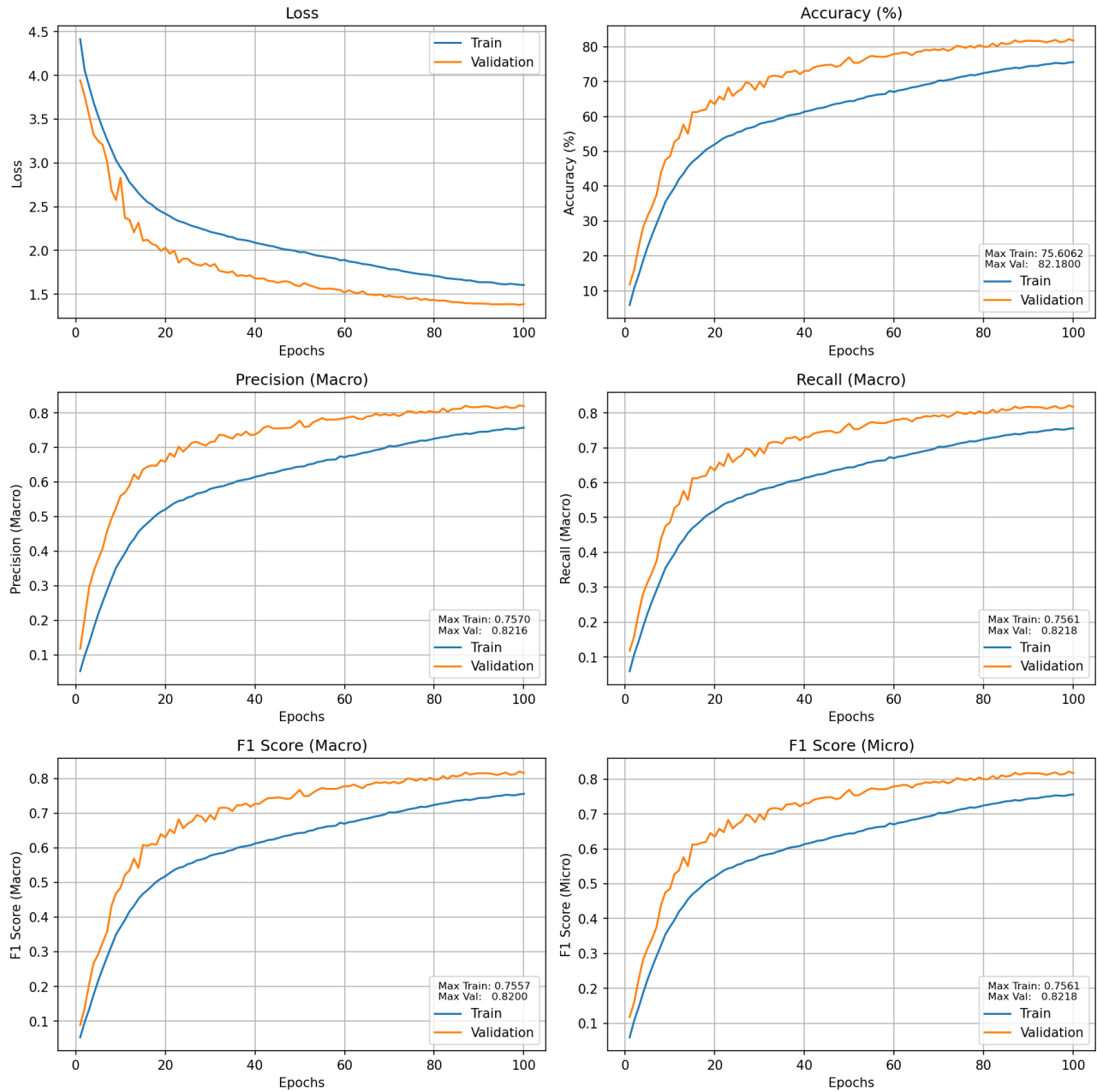


Figure 1: Training and validation curves for the Baseline model (PyTorch `nn.BatchNorm2d`).

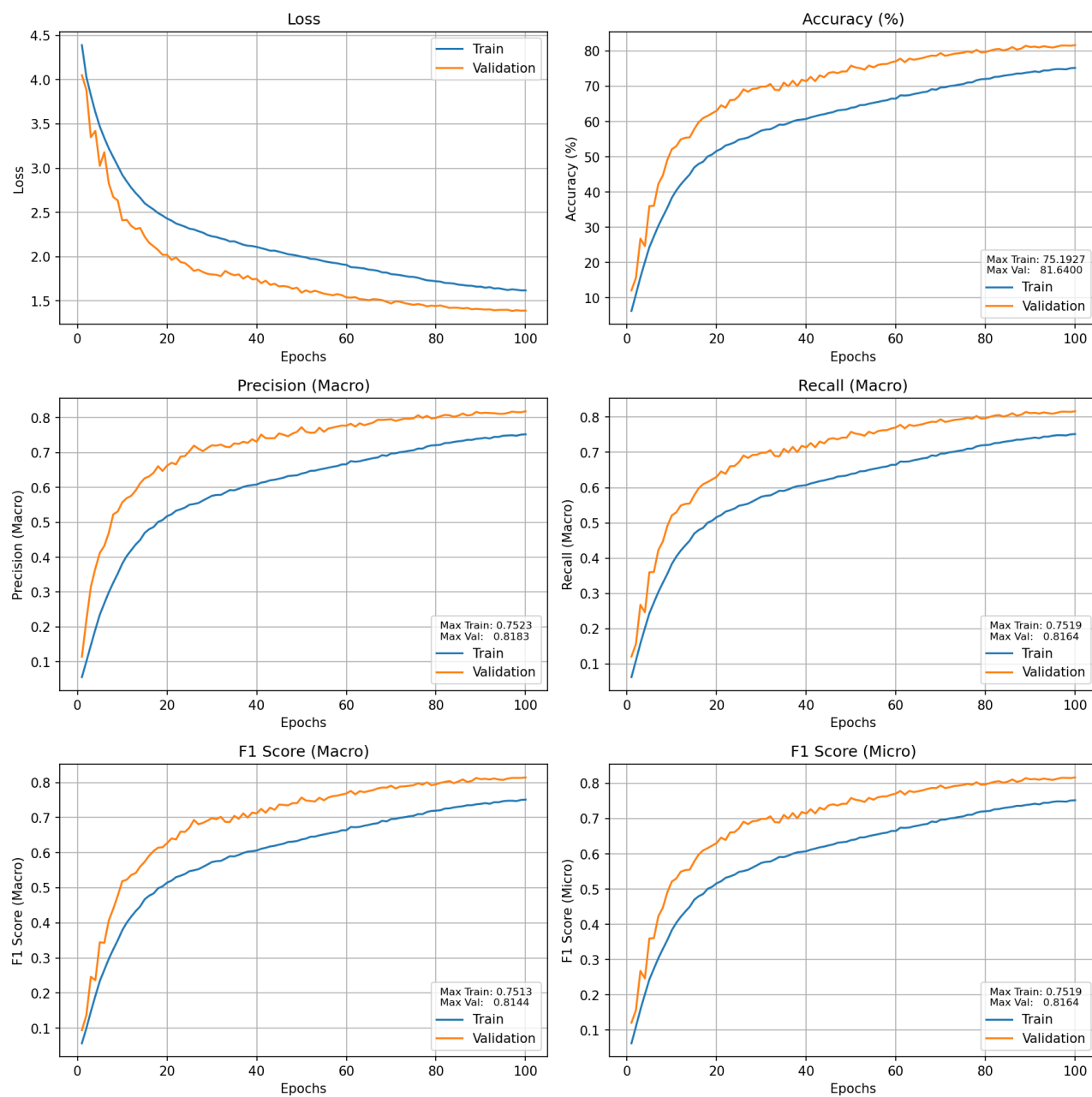


Figure 2: Training and validation curves for the custom Batch Normalization (BN) variant.

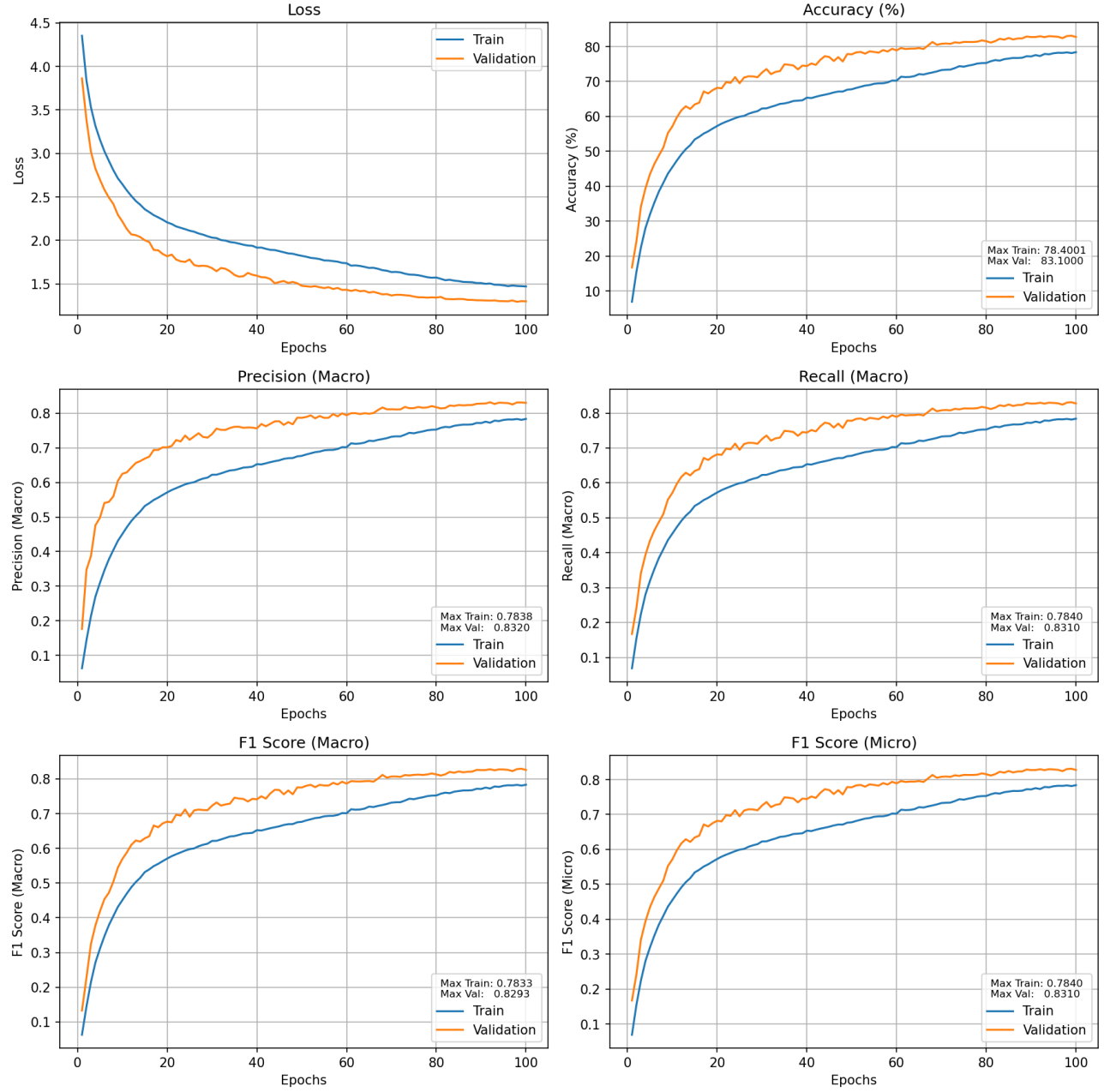


Figure 3: Training and validation curves for the Batch-Instance Normalization (BIN) variant.

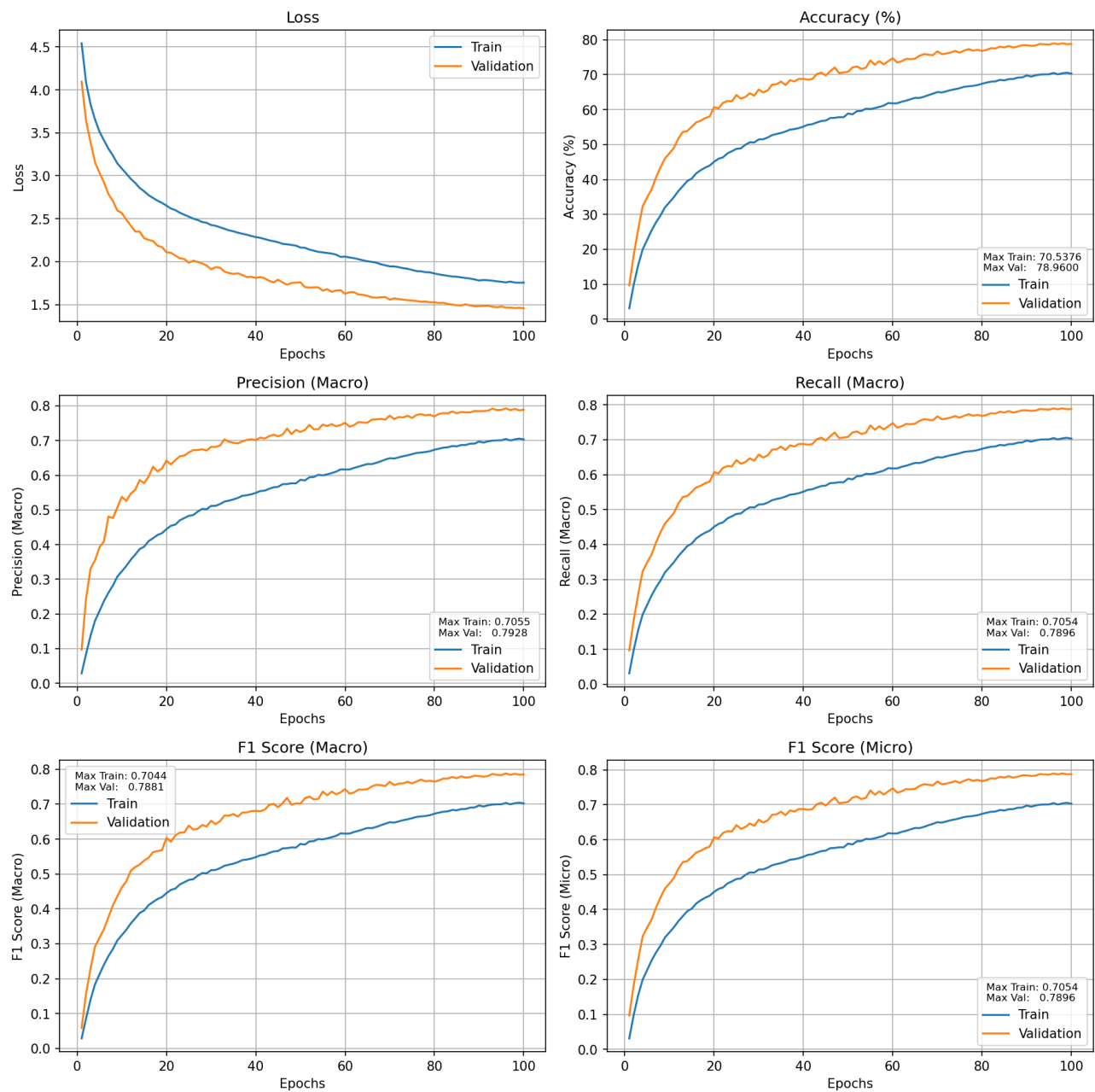


Figure 4: Training and validation curves for the Instance Normalization (IN) variant.

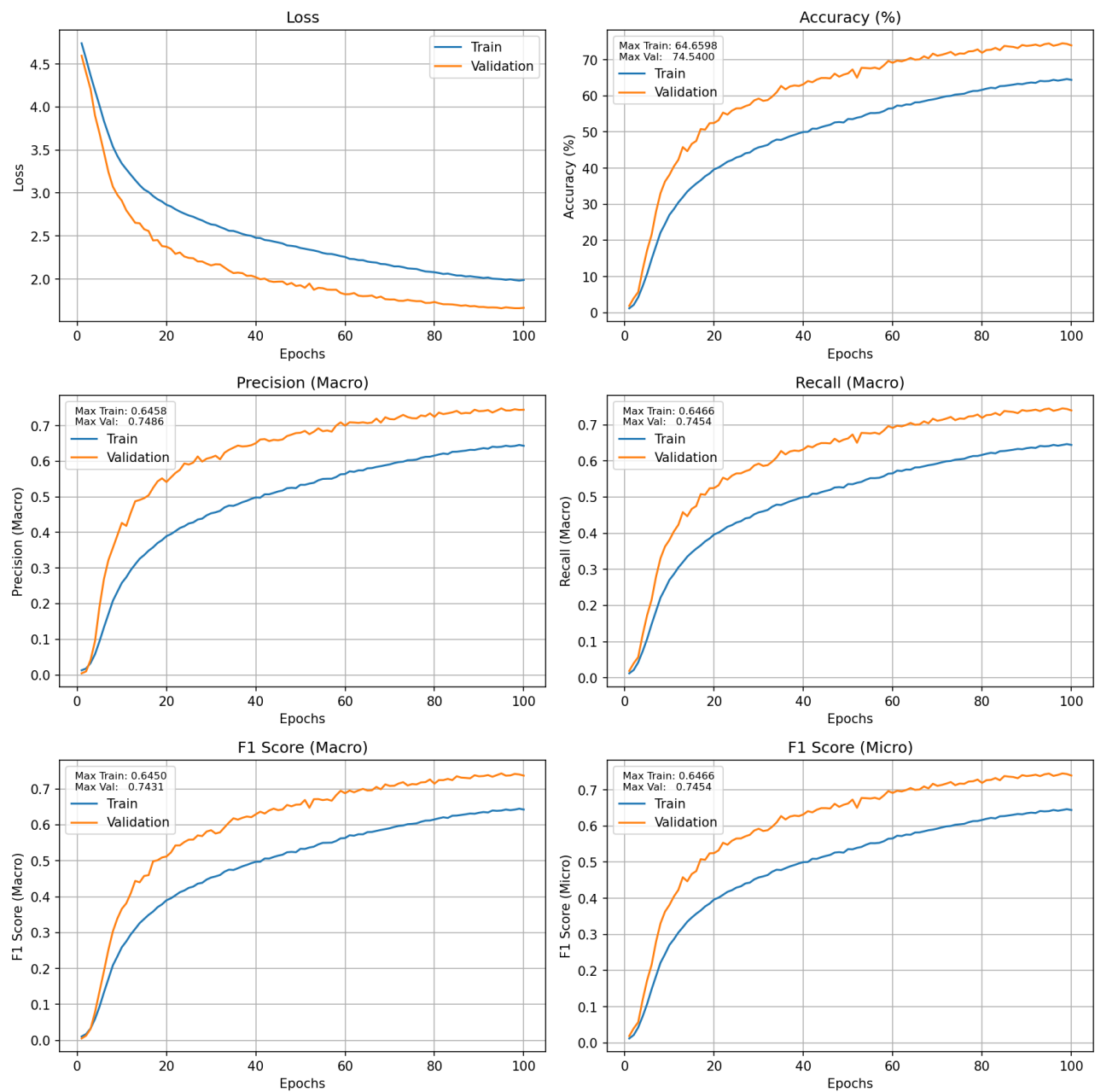


Figure 5: Training and validation curves for the Layer Normalization (LN) variant.

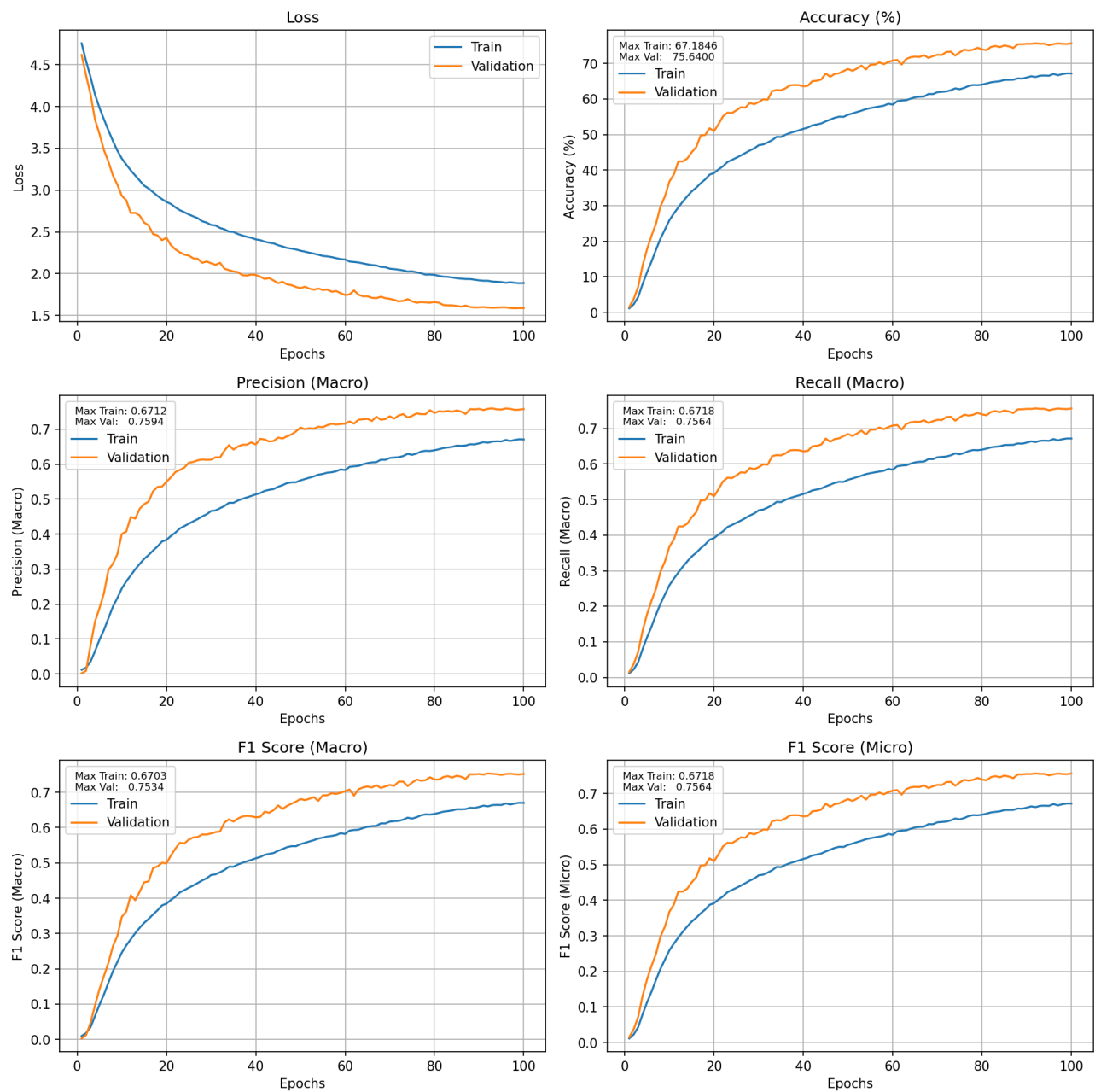


Figure 6: Training and validation curves for the Group Normalization (GN) variant.

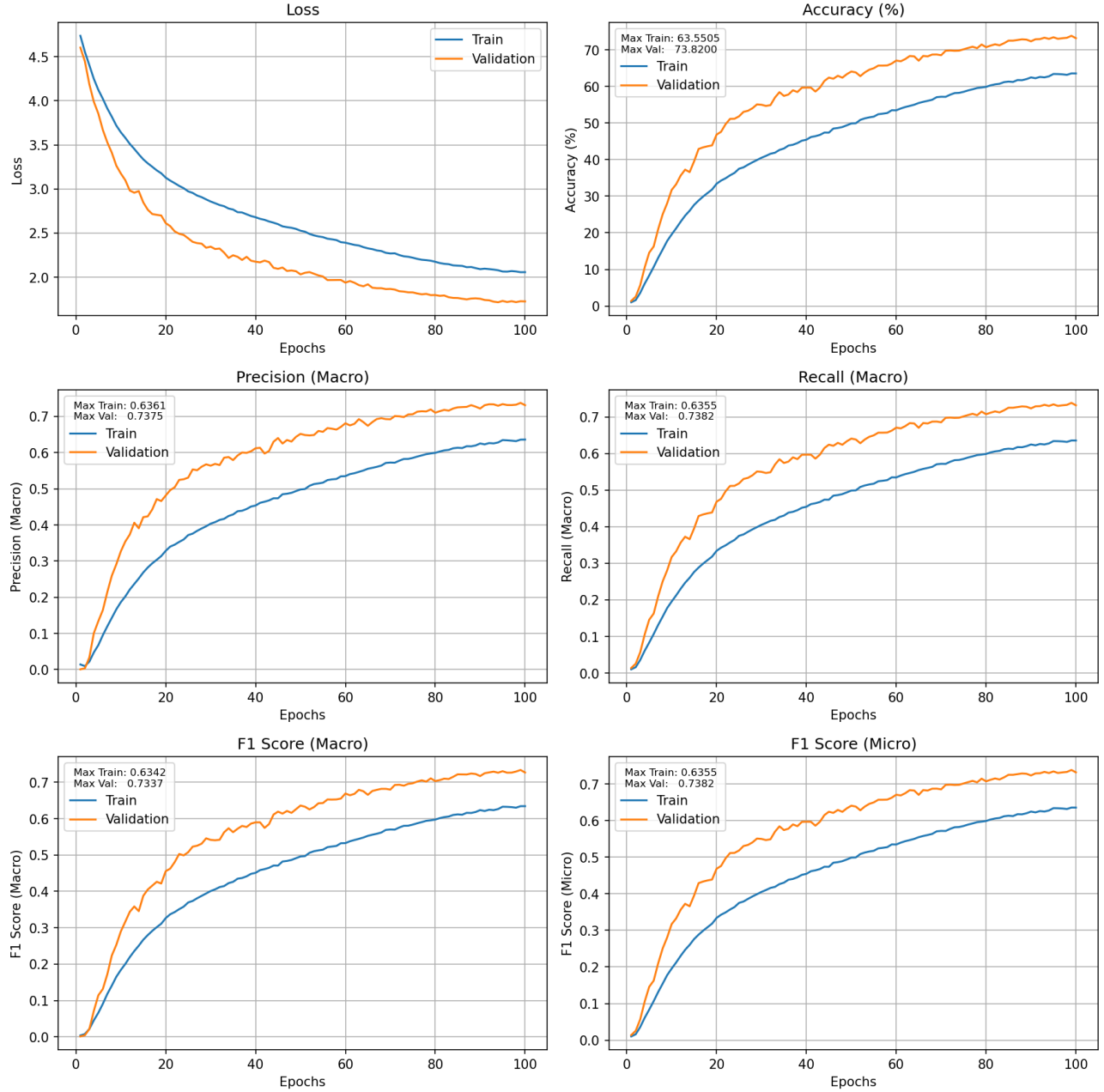


Figure 7: Training and validation curves for the No Normalization (NN) variant.

4.2 Quantitative Results

Table 2 summarizes the best validation metrics achieved by each variant over 100 epochs. Since Micro-F1 equals accuracy in the balanced evaluation setting used here, we report only Accuracy and Macro-F1 for conciseness.

Table 2: Best metrics across all normalization variants (100 epochs, batch size 128)

Variant	Train Acc. (%)	Train Macro-F1	Val Acc. (%)	Val Macro-F1
Baseline (<code>nn.BatchNorm2d</code>)	75.60	0.7557	82.18	0.8200
BN (Custom)	75.19	0.7513	81.64	0.8144
BIN	78.40	0.7833	83.10	0.8293
IN	70.53	0.7044	78.96	0.7881
LN	64.65	0.6450	74.54	0.7431
GN	67.18	0.6703	75.64	0.7534
NN	63.55	0.6342	73.82	0.7337

4.3 Analysis

4.3.1 Baseline vs Custom Batch Normalization

The custom BN implementation achieves 81.64% accuracy compared to the Baseline’s 82.18%, a difference of only $\sim 0.5\%$. This near-identical performance confirms that our custom implementation is correct and consistent with PyTorch’s built-in `nn.BatchNorm2d`. The marginal difference can be attributed to minor floating-point differences in running statistics computation rather than any fundamental implementation divergence. The loss and convergence curves in Figures 1 and 2 are visually almost indistinguishable, further validating this.

4.3.2 Batch-Instance Normalization

BIN achieves the highest validation accuracy at 83.10%, outperforming even the Baseline. This is consistent with findings in the original BIN paper: the learnable per-channel ρ parameter allows the model to adaptively blend batch-level and instance-level statistics, selecting whichever is more useful for each feature map. For image classification tasks, some channels benefit from global batch statistics (for stable feature learning) while others benefit from local instance-level normalization (for style invariance), and BIN is able to exploit both.

4.3.3 Instance Normalization

IN achieves 78.96% accuracy, placing it below BN but well above LN, GN, and NN. Instance normalization normalizes per sample per channel, removing instance-specific contrast information. While this property is beneficial in style transfer tasks, it is less ideal for classification where global channel statistics carry discriminative information. Nonetheless, it converges well and benefits from the stability that any form of normalization provides.

4.3.4 Layer Normalization and Group Normalization

Both LN (74.54%) and GN (75.64%) trail BN and IN noticeably. Interestingly, looking at the training curves in Figures 5 and 6, both schemes converge rapidly in the first 5–10 epochs, initially tracking close to BN, but fall behind in later epochs. LN normalizes across the entire (C, H, W) volume per sample, which is effective in sequence models but suboptimal for convolutional features where channel-wise statistics carry more meaningful structure. GN with 32 groups provides a reasonable middle ground, and its slight edge over LN is expected since it preserves some channel grouping structure. Both methods are better suited to settings with small batch sizes, which is not the bottleneck in our experiments.

4.3.5 No Normalization

NN achieves the lowest accuracy at 73.82%. Crucially, looking at Figure 7, the training and validation curves for NN have not converged by epoch 100, unlike all other variants which show clear plateauing behavior. This directly illustrates the role of normalization in stabilizing and accelerating convergence: without it, the internal covariate shift across layers slows down training significantly. The large gap between train accuracy (63.55%) and validation accuracy (73.82%) also suggests that the model without normalization struggles to learn robust representations, as the optimization landscape is more difficult to navigate.

4.3.6 Overall Observations

All normalization schemes produce qualitatively similar curve shapes, with the primary differences being in the final convergence values rather than in the shape of training dynamics. The ordering $\text{BIN} > \text{Baseline} \approx \text{BN} > \text{IN} > \text{GN} > \text{LN} > \text{NN}$ is consistent with the theoretical properties of each scheme. Normalization generally provides a clear benefit both in final performance and training stability. The key takeaway is that the choice of normalization scheme matters, with adaptive methods like BIN offering the most flexibility and performance for standard image classification at this batch size.

5 Batch Size Sensitivity

5.1 Setup

We retrain the BN and GN variants with a batch size of 8 to empirically verify the commonly cited claim that Group Normalization is more robust to small batch sizes than Batch Normalization. Both models are trained for 50 epochs. To account for the smaller batch size, we scale the learning rate accordingly, using a Cosine Annealing schedule from 10^{-4} to 10^{-5} instead of the standard 10^{-3} to 10^{-4} . All other hyperparameters remain identical to the standard configuration.

5.2 Training Curves

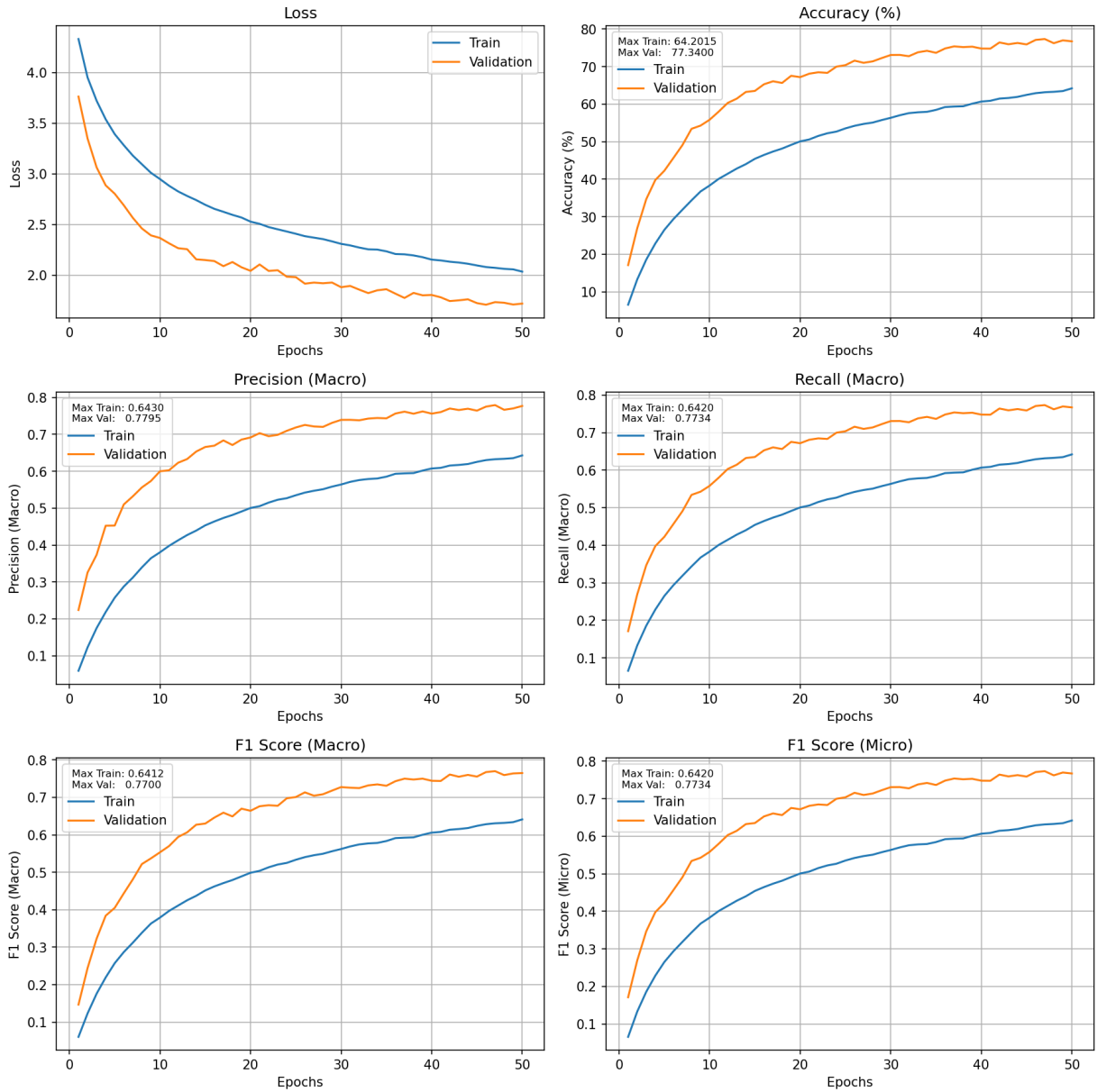


Figure 8: Training and validation curves for BN with batch size 8.

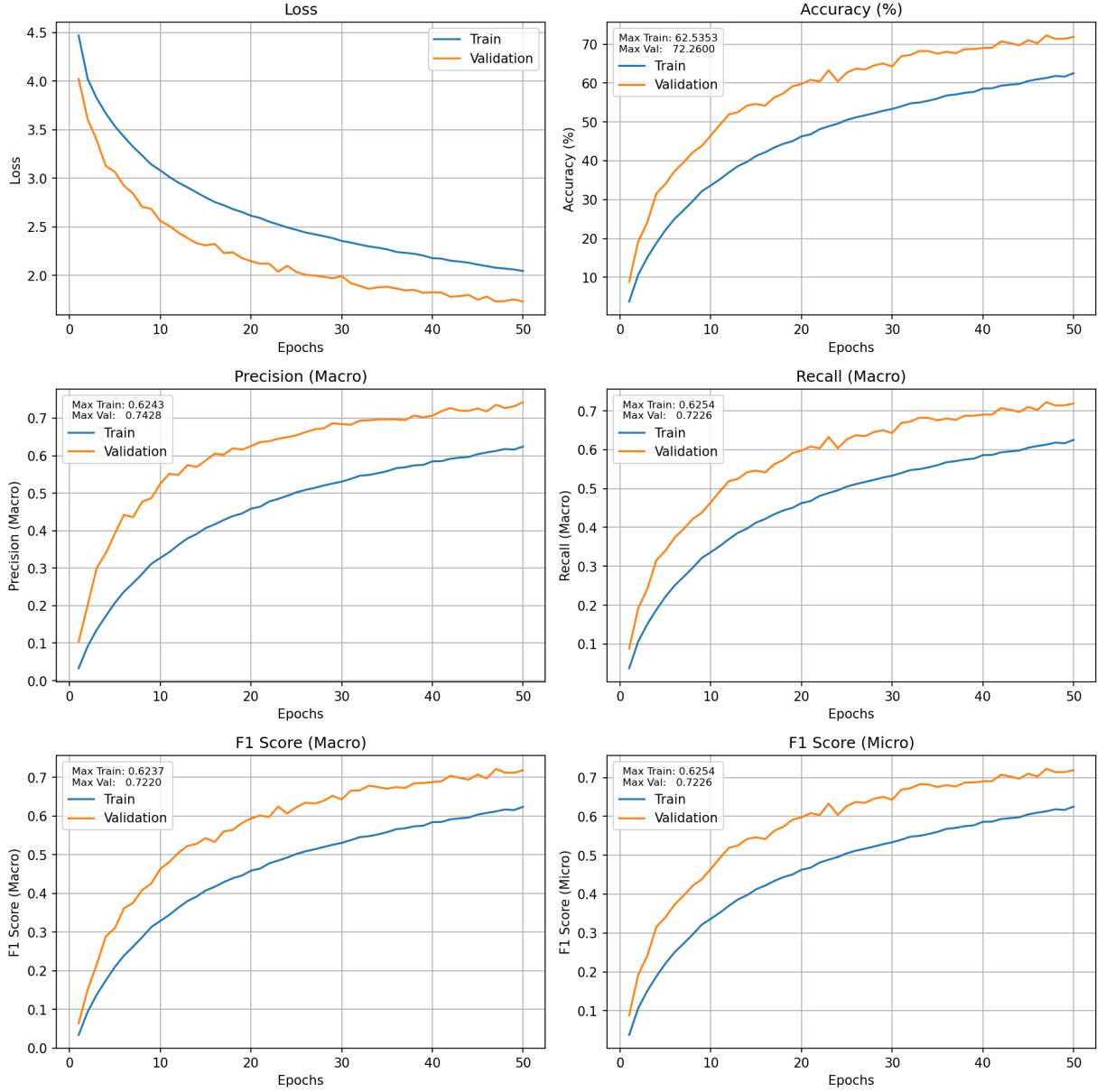


Figure 9: Training and validation curves for GN with batch size 8.

5.3 Quantitative Results

Table 3: Batch size sensitivity: BN vs GN at batch sizes 128 and 8

Variant	Batch Size	Train Acc. (%)	Train F1	Val Acc. (%)	Val F1	Val Acc. Drop
BN	128	75.19	0.7513	81.64	0.8144	↓ 4.30
BN	8	64.20	0.6412	77.34	0.7700	
GN	128	67.18	0.6703	75.64	0.7534	↓ 3.38
GN	8	63.53	0.6237	72.26	0.7220	

5.4 Analysis

- BN degrades by approximately 4–5% when the batch size is reduced from 128 to 8, while GN degrades by only approximately 3%, confirming that GN is more robust to small batch sizes.
- The reason for BN’s sensitivity is fundamental to how it operates: at batch size 8, the mini-batch mean and variance are estimated from very few samples, making them noisy and unreliable proxies for the true population statistics. This noise propagates through the normalization and corrupts the gradient signal, leading to less stable training and poorer generalization.
- GN computes statistics independently per sample within groups of channels, making it entirely independent of batch size. Its normalization behavior at batch size 8 is identical to that at batch size 128, which is why the performance degradation is smaller. The residual drop of $\sim 3\%$ in GN is likely attributable to fewer gradient updates per epoch at batch size 8 relative to the 50-epoch budget rather than any instability in the normalization itself.
- The train-val gap increases noticeably at batch size 8 for both variants. For BN, the train accuracy drops from 75.19% to 64.20% while val only drops from 81.64% to 77.34%, widening the gap significantly. A similar pattern holds for GN. This suggests that at batch size 8, the noisy gradient updates make it harder for the model to fit the training data, while validation performance is relatively more preserved due to the use of running statistics at inference for BN and the inherent stability of GN.
- Even at batch size 8, BN (77.34%) still outperforms GN (72.26%) in absolute terms, suggesting that BN’s advantage at standard batch sizes is large enough that GN does not overtake it even under these constrained conditions.

6 Grad-CAM Visualizations

We use Grad-CAM to analyze model behavior on the best and worst performing classes using the BIN model, which achieved the highest validation accuracy of 83.10% across all normalization variants. Specifically, we use the best saved checkpoint of the BIN model, i.e., the weights corresponding to the epoch with the highest validation accuracy during training. For each class, we visualize 5 correctly classified and 5 incorrectly classified images, with gradients computed with respect to the ground-truth class label.

6.1 Visualizations

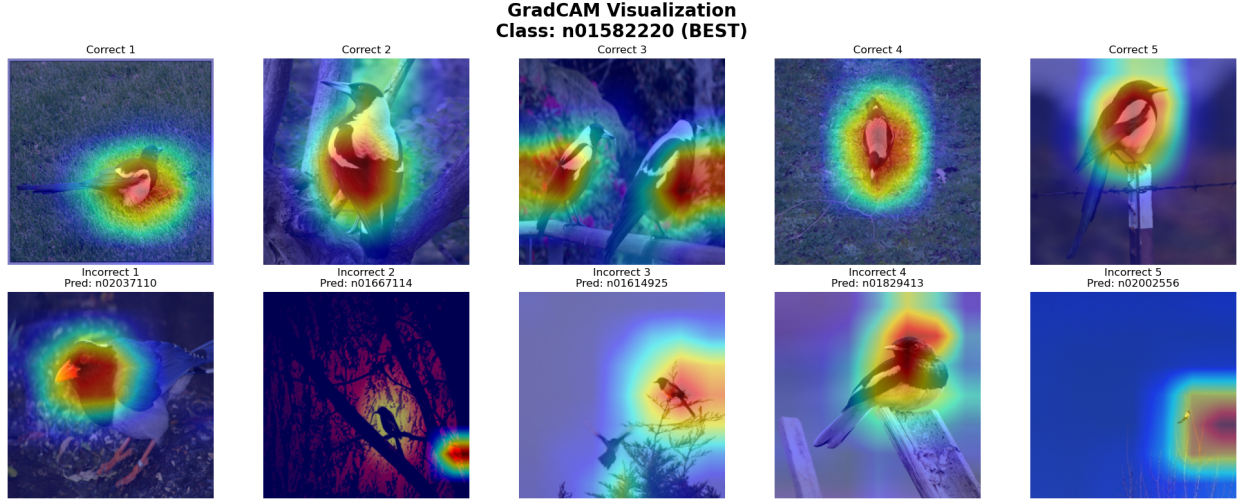


Figure 10: Grad-CAM visualizations for class n01582220 (one of the best performing classes).

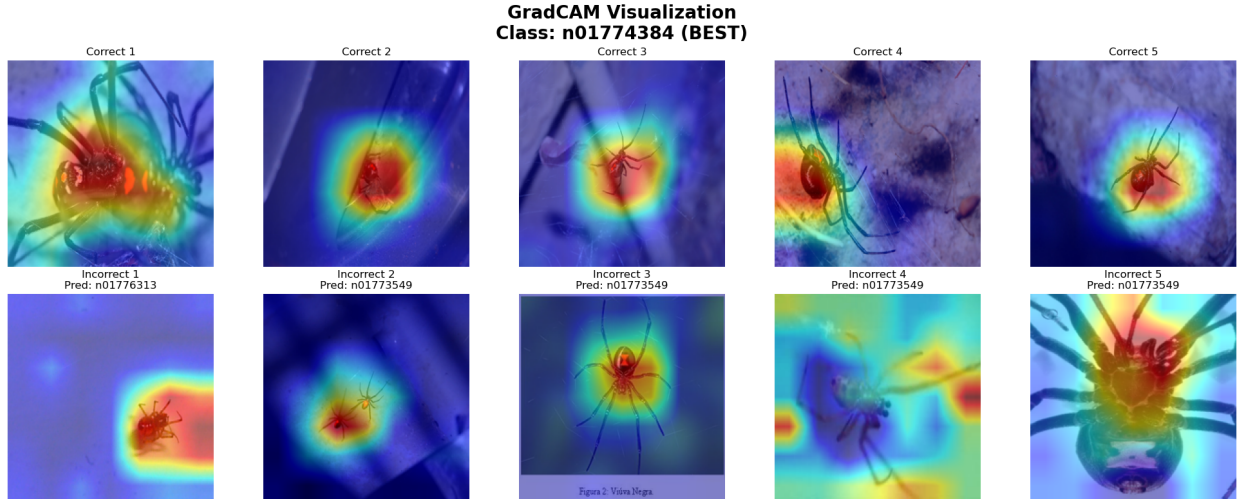


Figure 11: Grad-CAM visualizations for class n01774384 (one of the best performing classes).

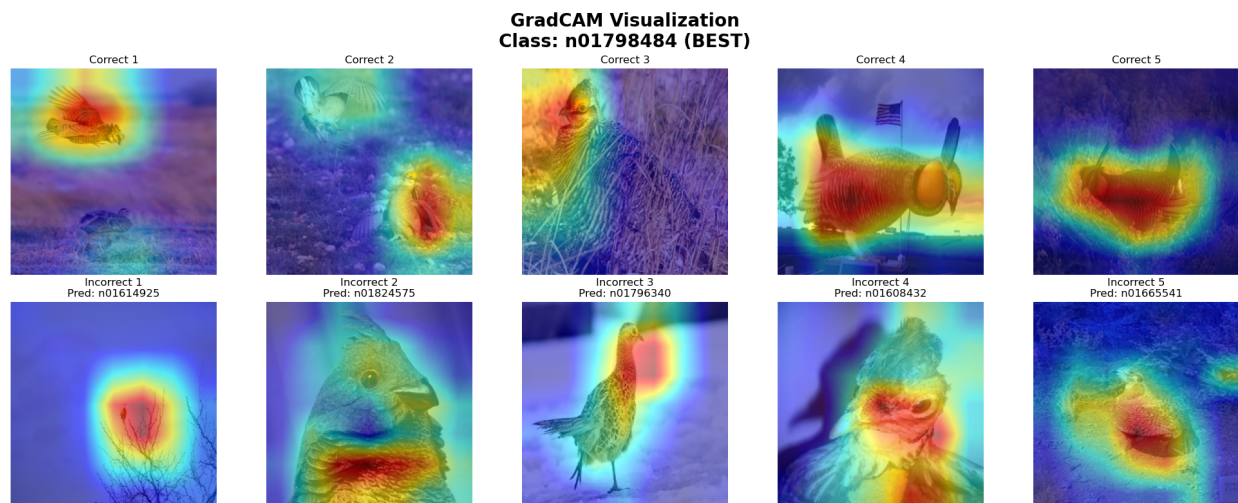


Figure 12: Grad-CAM visualizations for class n01798484 (one of the best performing classes).

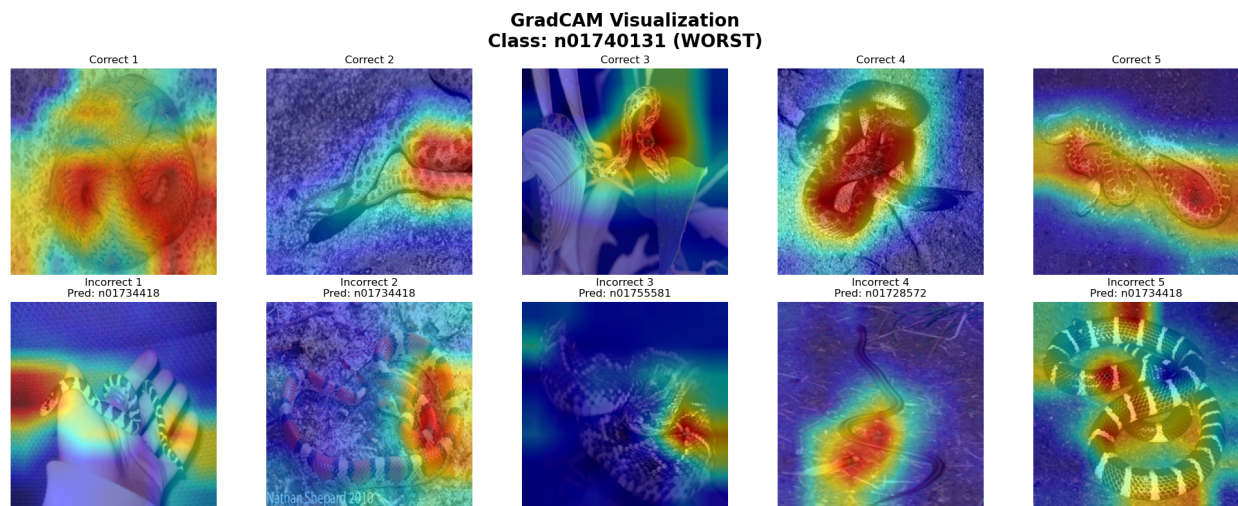


Figure 13: Grad-CAM visualizations for class n01740131 (one of the worst performing classes).

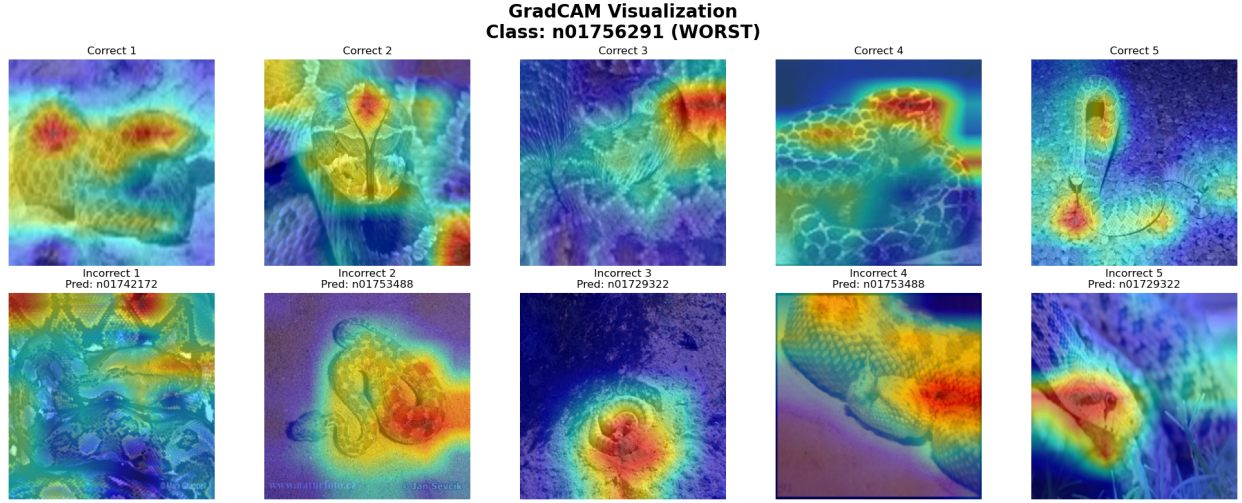


Figure 14: Grad-CAM visualizations for class n01756291 (one of the worst performing classes).

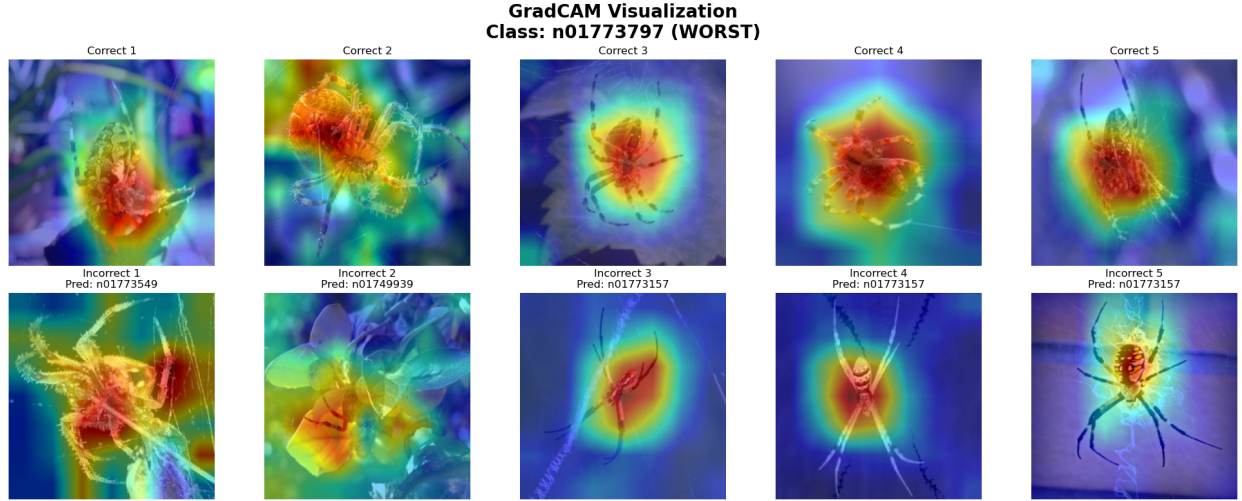


Figure 15: Grad-CAM visualizations for class n01773797 (one of the worst performing classes).

6.2 Analysis

6.2.1 Best Performing Classes

Correct Predictions:

- For the bird classes (n01582220 and n01798484), activations are tightly localized on the torso and distinctive plumage of the bird, with the background largely suppressed.
- For the spider class (n01774384), activations consistently concentrate on the abdomen and body across all five correct predictions, indicating a robust and stable learned representation.
- In general, correct predictions across all best performing classes show focused, spatially compact activation maps centered on the most discriminative part of the subject.

Incorrect Predictions:

- For n01582220, Incorrect 2 is a silhouette against a dark background where color and texture features are unavailable, causing activations to scatter rather than localize, leading to a wrong prediction.
- For the remaining incorrect bird images (n01582220 and n01798484), the activations are still well-localized on the bird body, but predictions are to other bird species, indicating fine-grained inter-class confusion rather than a localization failure.
- For n01774384, incorrect predictions are almost exclusively to a single other spider class (n01773549), and activations remain focused on the spider body even for wrong predictions, again pointing to fine-grained confusion as the primary failure mode.

6.2.2 Worst Performing Classes

Correct Predictions:

- Even for correctly classified images, activations are noticeably more diffuse and spread across larger regions compared to the best performing classes.
- For the snake classes (n01740131 and n01756291), the model attends to scale texture and body shape across the full extent of the animal, without a single strongly discriminative focal region, consistent with the challenge of distinguishing species that differ in subtle patterns across the entire body.
- For n01773797, correct predictions show activations spread across the full spider body including legs, suggesting the model relies on a combination of body structure and leg arrangement rather than any single compact feature.

Incorrect Predictions:

- A consistent failure mode across all three worst performing classes is intra-category confusion: incorrect predictions are almost exclusively to other species within the same broad category (snakes confused with snakes, spiders confused with spiders).
- For n01740131, most incorrect predictions are to n01734418 (another snake species). For n01756291, incorrect predictions are to n01742172, n01753488, and n01729322 (all snakes). For n01773797, incorrect predictions are to n01773549 and n01773157 (other spiders).
- Grad-CAM activations for incorrect predictions in these classes are more spread out and less confident compared to the tight localization seen in correct predictions, suggesting the model is uncertain about which region is specifically discriminative for the target class.
- Some incorrect predictions show activations on the correct general region but with diffuse maps covering surrounding context, suggesting the model is picking up on background or contextual cues shared between similar species rather than the subject itself.

6.2.3 General Observations

- Correct predictions consistently show more focused and spatially concentrated activations compared to incorrect ones across all classes.
- The best performing classes benefit from distinctive and visually separable features, enabling tight localization and reliable classification.

- The worst performing classes all suffer from fine-grained intra-category confusion, where the model attends to the right region but cannot distinguish between visually similar species within the same category.
- Challenging imaging conditions such as silhouettes, extreme close-ups, or poor contrast consistently lead to scattered activation maps and incorrect predictions, indicating that the model relies heavily on texture and color features.

Conclusion

In this work, we implemented ResNet-18 from scratch and systematically evaluated six normalization schemes on a 100-class ImageNet subset. The results reveal a clear hierarchy: BIN achieves the best accuracy (83.10%) by adaptively blending batch and instance statistics, while the standard BN variants closely follow. IN performs reasonably well, whereas LN and GN trail noticeably despite their theoretical appeal, as neither is optimally suited to the channel-structured features of convolutional networks at standard batch sizes. Removing normalization entirely (NN) yields the worst performance and visibly fails to converge within 100 epochs, underscoring how critical normalization is for stable optimization in deep residual networks.

The batch size sensitivity experiment confirms the well-known vulnerability of BN to small batches: reducing the batch size from 128 to 8 causes a $\sim 4.3\%$ drop in BN’s validation accuracy versus only $\sim 3.4\%$ for GN, validating GN as the more robust choice in memory- or data-constrained regimes. Grad-CAM analysis further reveals that the model’s failure modes are predominantly due to fine-grained intra-category confusion rather than localization errors – the best performing classes exhibit tightly focused activations on discriminative regions, while the worst performing classes (visually similar snake and spider species) show diffuse maps with no single strongly discriminative anchor.

Overall, the experiments highlight that normalization scheme selection should be driven by the operating batch size and task structure, with adaptive approaches like BIN offering the most flexibility for standard classification settings.